Methodology Memo

Behavioural Risk Intelligence for Agentic Payments

Bloomsbury Tech – Agentic Payments Hackathon Build

April 25, 2026

Abstract

This memo explains the fraud-detection methodology behind the current `agentic-payments` prototype. It surveys how fraud detection is typically done in Web2 payments, ecommerce, and digital banking; identifies which ideas transfer to agentic payment rails; and specifies the exact feature, clustering, scoring, explanation, and evaluation methods implemented in the repository. The central claim is that agentic payment risk is primarily a statistical and graph-learning problem, not a large-language-model problem: agents leave machine-shaped behavioural fingerprints in timing, gas optimization, method calls, counterparties, and coordination structure.

1 Executive Summary

Modern fraud systems combine four ingredients: (i) dense event data, (ii) behavioural features, (iii) supervised or semi-supervised risk models, and (iv) an operational decision layer for review, step-up verification, blocking, or liability shift. Stripe Radar, Shopify Fraud Analysis/Protect, Riskified, Sift, Forter, Sardine, Monzo, and Revolut differ in surface area, but they all converge on that architecture: real-time scoring, network effects, rules plus machine learning, explainability for analysts, and feedback loops from chargebacks or confirmed fraud.

Agentic payments preserve this structure but change the signal. Human-centric features such as browser fingerprints, shipping address mismatch, CVC checks, email reputation, device biometrics, and human time-of-day patterns become weaker or unavailable. New agentic features become available: inter-arrival regularity, 24-hour uniformity, gas-to-basefee tightness, repeated calldata, contract/method concentration, late counterparty drift, and wallet-to-wallet coordination. Because labels are initially sparse, the first product should not pretend to be a fully supervised fraud classifier. It should be a behavioural risk intelligence layer: rank anomalous wallets, explain the signals, surface coordinated clusters, and improve as real labels arrive.

The repository currently implements:

- A synthetic labelled data generator for humans, deterministic agents, payment bots, compromised agents, and collusion rings.
- A public x402/Base data collector and decoder for EIP-3009-style USDC `transferWithAuthorization` settlements.
- A 36-feature behavioural fingerprint per payer wallet.
- Pairwise wallet clustering using counterparty overlap, method agreement, and timing co-movement.
- An eight-factor composite score calibrated against a low-volume baseline, mapped to risk tiers with human-readable explanations.

- A dashboard that presents triage, evidence, behaviour maps, and coordination rather than just a scatterplot.

2 What Fraud Detection Looks Like in Web2

2.1 Recurring Structure

The fraud literature has been consistent for two decades: fraud is rare, adaptive, adversarial, and operationally asymmetric. Bolton and Hand's classic review frames statistical fraud detection as the fallback once prevention fails, and emphasizes that fraudsters adapt to controls over time [1]. Ngai et al. classify financial fraud detection work around data mining families such as classification, clustering, outlier detection, regression, visualization, and social-network analysis [2]. Later credit-card fraud work stresses feature engineering, class imbalance, delayed labels, and concept drift [3, 4, 5].

In production systems, the common architecture is:

1. **Event capture:** payment, account, device, browser, network, order, chargeback, customer-service, and identity events.
2. **Feature computation:** real-time, near-real-time, and batch aggregations at transaction, account, device, card, merchant, and counterparty levels.
3. **Decisioning:** rules, supervised models, anomaly detectors, graph/link analysis, and risk thresholds.
4. **Action layer:** allow, review, challenge, step-up verification, block, reserve funds, cancel order, or reimburse/guarantee merchant.
5. **Feedback loop:** chargebacks, dispute outcomes, manual review decisions, confirmed scams, law-enforcement alerts, and analyst labels.

2.2 Stripe Radar

Stripe Radar is the closest Web2 analogue to our intended product. Stripe documents risk evaluation as a machine-learning score and risk level for each payment. Radar for Fraud Teams exposes a 0–99 risk score, with default thresholds around 65 for elevated risk and 75 for high risk [6]. Radar risk insights describe an adaptive AI model that uses hundreds of signals and data across Stripe's business network, and exposes some explanatory signals to analysts [8]. Radar rules then let merchants block or review payments according to score, risk level, CVC/AVS checks, and custom predicates [9]. Radar for Platforms extends the same logic from transaction fraud to connected-account risk: machine-learning risk scores, customized rules, analyst investigation, document/selfie verification, and actions such as reserves [10].

Methodological lesson: the product is not "a model." It is a risk operating system: score, explain, rule, review, and act. Our dashboard should therefore show queue, reason, action, and feedback hooks, not merely an embedding plot.

2.3 Shopify Fraud Analysis and Protect

Shopify's fraud analysis gives order-level fraud recommendations and indicators for online card orders, and its documentation states that the recommendations are powered by machine-learning algorithms trained on historical transactions across Shopify stores [11]. Shopify Protect adds a business model

layer: for eligible Shop Pay transactions, Shopify can cover protected fraudulent/unrecognized chargebacks and manage the dispute process [13].

Methodological lesson: ecommerce fraud systems are not just score providers; they change merchant behaviour by saying which orders are safe to fulfil, which require verification, and where liability sits. Our equivalent actions are: allow, monitor, step-up auth, throttle/sandbox agent, freeze, or submit dispute/AML evidence.

2.4 Chargeback-Guarantee and Risk-Network Vendors

Riskified describes real-time approve/decline decisions from machine-learning models over hundreds of transaction features, a merchant network with over a billion historical transactions, and an aligned-incentive chargeback guarantee [17]. Its product pages emphasize data collection, device/browser behavioural insight, merchant-network linking, and explainable machine learning [18]. Sift advertises a global data network of roughly one trillion annual events, hundreds of brands, rulesets, and split-second decisioning [19]. Forter emphasizes identity protection and a global identity graph across more than a billion online identities [20]. Sardine combines consortium data, supervised ML, device intelligence, behaviour biometrics, graph connections, rules, and case management [21, 22].

Methodological lesson: the defensible asset is a labelled network of behaviour, not a one-off classifier. Our wedge should start with wallet-level behavioural features and grow into an agent reputation graph.

2.5 Digital Banks and Real-Time Payment Fraud

Digital banks make the latency problem explicit. Monzo's fraud platform separates feature computation from control execution, uses just-in-time, near-real-time, and batch features, and records control metadata to measure effectiveness and false positives [23]. Monzo's ML stack also emphasizes reusable feature engineering and feature monitoring [24]. Revolut's scam-detection feature uses machine learning to detect likely scam payments, decline high-risk payments, send users through an intervention flow, ask additional questions, and escalate to fraud specialists [25].

Methodological lesson: risk models should be measured as controls on a payment hot path. The key product metrics are not only AUC or accuracy, but latency, recall at tolerable false-positive rate, false-positive cost, queue volume, and loss prevented.

3 Why Agentic Payments Are Different

Agentic payments are not merely card payments with a new front end. Protocols such as x402 make payments HTTP-native and machine-executable. Coinbase's documentation describes facilitators that verify signed payment payloads and settle them on-chain, with hosted support for stablecoin payments and EIP-3009 compatible tokens on Base/Polygon and SPL tokens on Solana [26]. x402's flow lets a server respond with HTTP 402, the client resubmit with a payment payload, and a facilitator verify and settle before the server returns the requested resource [27]. This yields a new machine-to-machine payment substrate.

The fraud signal changes in four ways:

1. **Human context weakens.** Many Web2 signals – browser history, shipping address, email reputation, human typing patterns, and shopper/customer-service history – are less relevant for autonomous agents.

2. **Machine regularity strengthens.** Agents are more likely to transact at high frequency, on cron-like intervals, across all hours, with repeated method selectors and narrow value bands.
3. **On-chain settlement exposes execution details.** On Base/x402 we can see facilitator/signer addresses, USDC contract calls, calldata method selectors, gas used, gas price, payer, recipient, and settlement time.
4. **Risk propagates through agent networks.** Compromised agents can call other agents, APIs, wallets, or merchants. This motivates graph and cascade methods rather than purely per-transaction scoring.

Therefore the product should begin as *behavioural risk intelligence for agent traffic*: wallet fingerprints, anomaly factors, coordination clusters, and interpretable triage. Fully supervised fraud labels can arrive later via chargebacks, confirmed compromised wallets, facilitator declines, AML alerts, merchant disputes, and manual analyst review.

4 Data Model

4.1 Canonical Transaction Row

The repository standardizes both synthetic and real x402 data to the following schema:

Field	Meaning
`block_time_s`	Seconds from window start
`block_time`	Timestamp when available
`from_addr`	Payer wallet, agent wallet, or synthetic user wallet
`to_addr`	Recipient, merchant, contract, or counterparty
`value_usd`	Transaction value normalized to USD/USDC units
`gas_used`	Execution gas used
`gas_price_gwei`	Gas price converted to gwei
`method_id`	EVM function selector, e.g. `0xe3ee160e`
`success`	Boolean transaction success flag

Synthetic data additionally has ground-truth labels: `human_random`, `agent_arb_deterministic`, `agent_payment_bot`, `agent_compromised`, and `collusion_ring`. Public x402/Base data is decoded from facilitator/signer transactions to Base USDC. In the current pull, we normalize 7,512 facilitator-address transactions, decode 437 x402/EIP-3009 USDC payment rows, and score 116 payer wallets.

4.2 Why Synthetic Data Is Legitimate Here

Synthetic data is not evidence that real fraud exists. It is a controlled instrumentation test. We need to know whether the pipeline can recover known policies:

- Humans: sparse, diurnal, heterogeneous values and counterparties.
- Deterministic agents: periodic, 24-hour, gas-tight, low method entropy.
- Payment bots: bursty but consistent recipients and transfer methods.
- Compromised agents: late drift to new counterparties and higher burst rate/value changes.
- Collusion rings: multiple wallets transacting with the same target in tight time windows.

The public x402 overlay then answers a separate question: does the schema and feature machinery work on real settlement data?

5 Feature Engineering

For every wallet w, let $T_w = \{t_1, \ldots, t_n\}$ be sorted transaction times, C_w counterparties, M_w method selectors, V_w values, and G_w gas observations. We compute 36 features in six families.

5.1 Temporal Features

Inter-arrival times are

$$\Delta_i = t_i - t_{i-1}, \quad i = 2, \ldots, n.$$

The coefficient of variation is

$$\mathrm{CV}_\Delta(w) = \frac{\mathrm{std}(\Delta)}{\mathrm{mean}(\Delta) + \epsilon}.$$

Near-zero values are cron-like; values near one resemble Poisson-like arrival; large values indicate burst/drift behaviour. We also compute Shannon entropy of log-binned inter-arrivals:

$$H(\Delta) = -\sum_b p_b \log_2 p_b.$$

Time-of-day uniformity is measured as a Kolmogorov–Smirnov distance between the empirical hour-of-day CDF and a uniform 24-hour CDF:

$$D_{\text{uniform}}(w) = \sup_h |F_w(h) - F_U(h)|.$$

Low D_{uniform} is suspicious for agents because always-on agents can cover the full day evenly; humans tend to be diurnal. We also keep a KL-divergence from a canonical human time-of-day curve.

Burstiness follows Goh and Barabasi's coefficient:

$$B(w) = \frac{\sigma_\Delta - \mu_\Delta}{\sigma_\Delta + \mu_\Delta + \epsilon}.$$

5.2 Gas and Optimization Features

Agents are expected to optimize execution more tightly than humans. We compute mean and standard deviation of gas price, gas-to-basefee ratio, gas-used coefficient of variation, and a simple optimization score:

$$\text{gas_opt}(w) = \frac{1}{1 + \mathrm{std}(g/\text{basefee}) + \mathrm{CV}(\text{gas_used})}.$$

In synthetic data basefee is simulated; in real Base data, gas price and gas used are directly pulled from Blockscout.

5.3 Value, Counterparty, and Method Features

Value features include mean, median, standard deviation, log-standard deviation, entropy, and the share of transactions above the population 95th percentile. Counterparty features include number of unique counterparties, top-1 and top-3 concentration, entropy, repeat rate, and late-new-counterparty share:

$$\text{late_new}(w) = \frac{\#\{\text{transactions in final quartile to previously unseen } c\}}{\#\{\text{transactions}\}}.$$

This is the primary compromised-agent drift signal.

Method features include unique method count, dominant selector share, native transfer share, ERC-20 transfer share, and success rate. Repeated calls to a single selector, for example USDC `transferWithAuthorization`, are normal for a payment facilitator but can still be a useful concentration feature in combination with timing and counterparty features.

5.4 Coordination Features

We compute the peak count in any 60-second window, burst windows per hour, mean inter-burst minutes, and the number of distinct partner wallets with at least one transaction within a 60-second window of the wallet's own transactions. These features are intentionally cohort-dependent: they identify wallets whose behaviour is anomalous relative to the other wallets in the same observed window.

6 Wallet Coordination Graph

For each wallet w, define a profile

$$P_w = \{(c, m, t) : \text{wallet } w \text{ called counterparty } c \text{ with method } m \text{ at time } t\}.$$

For wallet pair (i, j), compute:

$$\begin{aligned}
O_{ij} &= \frac{|C_i \cap C_j|}{|C_i \cup C_j|}, \\
M_{ij} &= \frac{1}{|C_i \cap C_j|} \sum_{c \in C_i \cap C_j} \mathbf{1}\{M_i(c) \cap M_j(c) \neq \varnothing\}, \\
T_{ij} &= \frac{\#\{(t_i, t_j) : |t_i - t_j| \leq 60s,\ c_i = c_j\}}{\#\{(t_i, t_j) : c_i = c_j\}}.
\end{aligned}$$

The composite similarity is

$$S_{ij} = 0.3O_{ij} + 0.4M_{ij} + 0.3T_{ij}.$$

An edge is created when $O_{ij} \geq 0.10$ and $S_{ij} \geq 0.40$. Connected components with at least two connected wallets form coordination clusters. The pairwise loop is capped at the top 500 wallets by transaction count to keep the hackathon implementation tractable.

This mirrors the Polymarket wallet clusterer, replacing market overlap and direction agreement with counterparty overlap and method agreement.

7 Composite Risk Score

7.1 Baseline Calibration

Because labels are sparse in real x402 data, we use a low-volume baseline as a proxy for ordinary human/low-automation behaviour:

$$B = \{w : \text{tx_count}(w) \leq Q_{0.30}(\text{tx_count})\}.$$

For feature x, define baseline z-score:

$$z_x(w) = \frac{x(w) - \mu_B(x)}{\sigma_B(x) + \epsilon}.$$

Some factors are suspicious when high; others are suspicious when low. We map z-scores to a 0–100 one-sided suspicion score:

$$q_{\text{high}}(z) = 100 \cdot \frac{1}{1 + \exp(-1.5(z-1))}, \quad q_{\text{low}}(z) = q_{\text{high}}(-z).$$

The shift by one standard deviation keeps baseline observations below the medium-risk threshold and makes high scores require genuine tail behaviour.

7.2 Eight Factors and Weights

Factor	Direction	Weight
Inter-arrival anomaly	large deviation from baseline median	0.15
Time-of-day uniformity	unusually uniform, low KS	0.15
Gas tightness	unusually low gas variance	0.20
Counterparty concentration	high top-1 share with volume	0.10
Method concentration	high top selector share with volume	0.05
Burst intensity	high bursts per hour	0.10
Drift signal	high late-new-counterparty share	0.10
Coordination	high partner count in short windows	0.15

The composite score is

$$\text{risk}(w) = \text{clip}_{[0,100]}\left(\sum_{k=1}^{8} \alpha_k q_k(w)\right).$$

Risk tiers are:

$$\text{critical} \geq 75, \quad \text{high} \geq 50, \quad \text{medium} \geq 25, \quad \text{low} < 25.$$

This follows the production pattern used by Radar-like tools: scalar score, risk tier, rule/action layer, and analyst explanation.

7.3 Explanations

For each wallet, we sort the eight factor scores and generate a short explanation from the top three. Examples:

- "inter-arrival CV 0.05 (cron); 100% calls to one selector; 36% volume to single counterparty."
- "inter-arrival CV 1.45 (bursty); 98% volume to single counterparty; 100% calls to one selector."

This deliberately mirrors Web2 risk insights: the model should not simply declare a wallet risky; it should say what behavioural facts drove the score.

8 Embedding and Dashboard

The embedding is not the primary detector. It is an analyst visualization. We standardize numeric features and project them to two dimensions with UMAP, falling back to PCA if UMAP is unavailable. The dashboard now leads with a triage queue and selected-case evidence, then offers the embedding as a behaviour map. This follows the Polymarket dashboard pattern: overview cards, ranked tables, badges, tabs, factor bars, and evidence before visualization.

9 Current Results

9.1 Synthetic Stress Test

The synthetic run has 52,162 transactions and 188 wallet rows. The current calibration flags deterministic arbitrage agents and collusion rings as high risk, catches most compromised agents, and keeps human wallets out of the high-risk queue. This validates that the pipeline can recover the behaviours it is designed to detect.

9.2 Public x402/Base Overlay

The no-key public data collector currently pulls:

- 24 facilitators from x402.watch.
- 43 x402-related addresses/signers.
- 7,512 Base facilitator-address transactions.
- 437 decoded x402/EIP-3009 USDC payment rows.
- 116 payer wallets and 19 recipient wallets.

The public x402 payer-wallet scorer currently returns 65 medium-risk wallets, 51 low-risk wallets, and no high-risk claims. This is an important honest result: the real public slice is usable, but small and weakly labelled. It should be presented as a real-data overlay, not as proof of fraud.

10 Why These Methods, Not an LLM Classifier?

LLMs are useful for analyst summaries, documentation, and case narratives. The core fraud decision should be statistical and graph-based because:

- Fraud decisions require stable calibration and measurable false-positive rates.
- Payment risk depends on structured features and temporal/graph context.
- Regulators, merchants, and banks need explanations grounded in observed facts.
- New fraud modes produce drift and adversarial adaptation; feature and graph monitoring are easier to audit than generative outputs.

Future versions may use LLMs to generate analyst-readable case summaries, but the score should remain feature-grounded.

11 Evaluation Plan

The evaluation should mature in four stages:

1. **Synthetic recovery:** policy-level precision/recall for known deterministic agents, compromised agents, humans, and collusion rings.
2. **Real overlay sanity checks:** distribution of x402 features, duplicate/failed settlements, top recipients, facilitator drift, and stability of score thresholds over time.
3. **Weak labels:** Forta alerts, sanctioned/high-risk address lists, facilitator KYT declines, merchant disputes, and manually verified suspicious wallets.
4. **Supervised calibration:** cost-sensitive models, precision-recall curves, temporal holdouts, delayed-label handling, and active learning for analyst review.

Metrics should include precision at queue size, recall at fixed false-positive rate, expected loss avoided, review burden, latency, and threshold stability. For real-time deployment, the relevant comparison is not only model AUC but whether a control can safely run on the payment path.

12 Future Model Roadmap

1. **Feature store:** separate just-in-time features from near-real-time and batch aggregates, following the pattern used in digital-bank fraud systems.
2. **Graph model:** build a wallet–recipient–facilitator graph and use graph embeddings or GNNs once labels exist. CARE-GNN and related camouflage-resistant methods are relevant because fraudsters adapt both attributes and edges [28]. Blockchain AML work on Elliptic datasets demonstrates the utility of temporal transaction graphs [29, 30].
3. **Reputation layer:** convert wallet risk into portable agent, facilitator, merchant, and counterparty reputation signals.
4. **Decision policies:** map tiers to concrete actions: allow, monitor, step-up authorization, rate-limit, sandbox agent, freeze, request identity/delegation proof, or create dispute/AML evidence.
5. **Feedback loop:** store analyst decisions and post-event labels so the model compounds data advantage over time.

13 Generalising to Traditional Payment Rails

The current x402/Base implementation should be treated as the first rail adapter, not as the boundary of the product. The invariant is not a wallet; it is an actor producing payment behaviour. A production architecture should therefore separate rail-specific ingestion from a common event/entity graph.

13.1 Rail Adapters

Stripe is the easiest traditional-payment adapter because the Charge object already exposes risk outcomes, network status, decline/advice codes, `fraud_details`, card checks, payment-method fingerprints, refunds, disputes, and payment-method-specific details [7]. Stripe's Radar score and risk level should be stored as external model signals, not as ground truth.

Shopify is the natural ecommerce adapter. Its order-level fraud analysis surfaces AVS/CVV, IP, multiple-card attempts, and low/medium/high recommendations; the Order Risk API allows third-party risk assessments to be displayed back to merchants with accept/investigate/cancel recommendations [11, 12]. This makes Shopify a practical path to a private app demo: ingest orders and transactions, produce an agent-aware order-risk record, and write the explanation into the merchant workflow.

Visa and Mastercard direct integrations are not hackathon-easy because their risk-management products are partner/restricted surfaces. Public Visa material describes Visa Advanced Authorization and Visa Risk Manager as in-flight AI risk scoring plus rules/case-management tooling over VisaNet-scale attributes [14]. Mastercard describes Decision Intelligence and Transaction Fraud Monitoring as real-time/pre-authorization risk scoring products with rules, case management, business insights, and network intelligence [15, 16]. The practical route is through an issuer, acquirer, PSP, or payment facilitator that can provide authorization streams and chargeback labels.

13.2 Canonical Entity Graph

The wallet-centric schema should generalise to:

- **Payment events:** rail, time, amount, currency, status, payer, payee, instrument, merchant, order, channel, external score, and raw rail reference.
- **Entities:** customer/account, wallet, card token/fingerprint, bank account fingerprint, merchant, device/session, email/phone, IP/ASN, shipping/billing address hash, and agent identity where available.
- **Labels:** chargeback, confirmed fraud, issuer decline, Radar high/elevated/normal, Shopify high/medium/low, refund abuse, scam report, AML alert, and manual-review decision.

13.3 Feature Transfer

Most x402 features have direct traditional-payment analogues:

x402 feature family	Traditional-payment equivalent
Inter-arrival regularity	checkout/payment-attempt velocity and retry cadence
24-hour uniformity	always-on API or checkout automation
Gas tightness	repeated request/device/API-idempotency patterns
Counterparty concentration	merchant, SKU, recipient, or address concentration
Method concentration	card/wallet/BNPL/3DS strategy concentration
Burst intensity	card testing, checkout waves, promo/refund abuse
Late counterparty drift	ATO or compromised-agent shift to new merchants/SKUs
Coordination	linked accounts/cards/devices/IPs in tight windows

Traditional rails also add features that x402 lacks: AVS/CVV outcomes, 3DS/ECI signals, device/browser entropy, shipping/billing distance, merchant-category risk, issuer/network decline codes, capture delay, refund velocity, fulfilment status, and chargeback reason codes.

The venture-level framing is therefore: *behavioural risk intelligence for automated payment traffic across rails.* We start with x402 because it makes machine payments visible, then reuse the same fingerprinting and graph-risk engine for Stripe merchants, Shopify stores, PSPs, acquirers, issuers, and digital banks.

14 Limitations

- The synthetic labels are useful for pipeline validation but not proof of real-world fraud.
- The public x402 data pull is small and weakly labelled; no high-risk claim should be made from it yet.
- The low-volume baseline is a pragmatic proxy, not a guaranteed human baseline.
- Gas features need chain-specific normalization when moving beyond Base.
- Counterparty concentration can be benign for legitimate API/payment agents; it is only suspicious in combination with timing, drift, gas, and coordination.
- A production system needs delayed-label handling, threshold governance, monitoring, and analyst feedback.

15 Implementation Map

Method	Repository path
Synthetic policy generator	`src/ingest/synthetic.py`
Public x402 collector/decoder	`src/ingest/public_x402.py`
Behavioural features	`src/features/fingerprint.py`
Wallet clustering	`src/models/cluster.py`
Composite scoring	`src/models/score.py`
Embedding	`src/viz/embed.py`
Demo dashboard	`src/viz/dashboard.py`
Synthetic pipeline	`src/pipeline.py`
Public x402 pipeline	`src/pipeline_public.py`

References

[1] Richard J. Bolton and David J. Hand. Statistical Fraud Detection: A Review. *Statistical Science*, 17(3):235–255, 2002. https://projecteuclid.org/journals/statistical-science/volume-17/issue-3/Statistical-Fraud-Detection-A-Review/10.1214/ss/1042727940.pdf

[2] E. W. T. Ngai, Yong Hu, Y. H. Wong, Yijun Chen, and Xin Sun. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569, 2011. https://doi.org/10.1016/j.dss.2010.08.006

[3] Alejandro Correa Bahnsen, Djamila Aouada, Aleksandar Stojanovic, and Bjoern Ottersten. Feature Engineering Strategies for Credit Card Fraud Detection. *Expert Systems with Applications*, 51:134–142, 2016. https://doi.org/10.1016/j.eswa.2015.12.030

[4] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating Probability with Undersampling for Unbalanced Classification. IEEE Symposium Series on Computational Intelligence, 2015. https://doi.org/10.1109/SSCI.2015.33

[5] Fabrizio Carcillo, Andrea Dal Pozzolo, Yann-Ael Le Borgne, Olivier Caelen, Yannis Mazzer, and Gianluca Bontempi. Streaming active learning strategies for real-life credit card fraud detection. *International Journal of Data Science and Analytics*, 5:285–300, 2018. https://link.springer.com/article/10.1007/s41060-018-0116-z

[6] Stripe Documentation. Risk evaluation: risk score and risk outcomes. https://docs.stripe.com/radar/risk-evaluation

[7] Stripe Documentation. The Charge object. https://docs.stripe.com/api/charges/object

[8] Stripe Documentation. Risk insights. https://docs.stripe.com/radar/reviews/risk-insights

[9] Stripe Documentation. Fraud prevention rules. https://docs.stripe.com/radar/rules

[10] Stripe Documentation. Radar for Platforms. https://docs.stripe.com/connect/risk-management/risk-tooling

[11] Shopify Help Center. Fraud analysis. https://help.shopify.com/en/manual/fulfillment/managing-orders/protecting-orders/fraud-analysis

[12] Shopify Developer Documentation. Order Risk resource. https://shopify.dev/docs/api/admin-rest/2025-07/resources/order-risk

[13] Shopify. Shopify Protect. https://www.shopify.com/protect

[14] Visa. Visa Advanced Authorization + Visa Risk Manager: Managing fraud better, together. https://corporate.visa.com/content/dam/VCOM/corporate/solutions/documents/visa-risk-manager-cnp-global-infographic.pdf

[15] Mastercard. Decision Intelligence for fraud and risk management. https://www.mastercard.com/global/en/business/cybersecurity-fraud-prevention/risk-decisioning/decision-intelligence.html

[16] Mastercard. Transaction Fraud Monitoring. https://www.mastercard.com/global/en/business/cybersecurity-fraud-prevention/risk-decisioning/transaction-fraud-monitoring.html

[17] Riskified. Chargeback Guarantee and Fraud Protection. https://www.riskified.com/chargeback-guarantee/

[18] Riskified. Ecommerce fraud prevention solutions. https://www.riskified.com/fraud/prevention-solutions/

[19] Sift. Digital Fraud Prevention and Risk-Based Authentication. https://sift.com/

[20] Forter. Account Protection. https://www.forter.com/platform/identity-protection/

[21] Sardine Documentation. What Powers Sardine. https://docs.sardine.ai/guides/public/gettingstarted/whatpowerssardine

[22] Sardine. Agentic Financial Crime Platform for Fraud Prevention and AML. https://www.sardine.ai/

[23] Monzo. Building a reactive Fraud Prevention Platform. https://monzo.com/blog/build-a-reactive-fraud-prevention-platform

[24] Monzo. Monzo's machine learning stack. https://monzo.com/blog/2022/04/26/monzos-machine-learning-stack

[25] Revolut. Revolut launches AI feature to protect customers from card scams. https://www.revolut.com/en-US/news/revolut_launches_ai_feature_to_protect_customers_from_card_scams_and_break_the_scammers_spell/

[26] Coinbase Developer Platform. x402 Facilitator. https://docs.cdp.coinbase.com/x402/core-concepts/facilitator

[27] Coinbase Developer Platform. How x402 Works. https://docs.cdp.coinbase.com/x402/core-concepts/how-it-works

[28] Yingtong Dou, Zhiwei Liu, Li Sun, Yuening Deng, Hao Peng, and Philip S. Yu. Enhancing Graph Neural Network-based Fraud Detectors against Camouflaged Fraudsters. CIKM, 2020. https://arxiv.org/abs/2008.08692

[29] Mark Weber et al. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. KDD Workshop on Anomaly Detection in Finance, 2019. https://arxiv.org/abs/1908.02591

[30] Elliptic, MIT, and IBM researchers. The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset. 2024. https://arxiv.org/abs/2404.19109